TECHNICAL PROJECT REPORT

Vision-Guided Robot Pick System

AI-Powered Object Detection + Collaborative Robot Arm Control

Computer Vision
YOLOv8 Detection

Robotics
FAIRINO Arm

AI Agent
Gemini NLP

Visualization
Unity 3D

Project Overview

End-to-end robotic system integrating perception, intelligence, and control

Core Objective

This project integrates a **depth camera**, **YOLO object detection**, **hand-eye calibration**, and a **FAIRINO collaborative robot arm** — all controlled by natural language commands processed by a **Gemini AI agent**. A Unity 3D interface visualizes detected objects in real time and enables multilingual command input.

Key Integration Components

Depth Camera
Orbbec DaBai DCW RGB-D

AI Detection
YOLOv8 Object Detection

Calibration
SVD Hand-Eye Calibration

Robot Arm
FAIRINO Collaborative

AI Agent Capabilities

Natural Language Processing
Process commands in any language via Gemini API

Function Calling
Structured tool execution for robot control

Real-time Visualization
Unity 3D interface with live object tracking

Source Code

github.com/prof-lijar/vision-guided-robot-pick-system

System Architecture

End-to-end integration flow from perception to execution

Camera Input — RGB-D Stream → **YOLO Detection** — Object Recognition → **3D Position** — Depth Estimation → **Calibration** — SVD Transform → **Robot Control** — Motion Execution

ROS2 Middleware Backbone

Communication Layer

DDS-based publish-subscribe architecture for real-time data exchange between all system components

Topic-Based Messaging

Standardized topics: /camera/color/image_raw, /camera/depth/points, /detections, /robot_command

Multi-Platform Support

Ubuntu 22.04 native + WSL2 compatibility with ROS2 Humble Hawksbill

Unity 3D Visualization Layer

ROS-TCP-Connector

Seamless integration between ROS2 topics and Unity GameObjects for real-time visualization

Dynamic Object Spawning

3D primitives instantiated at detected positions with live coordinate updates

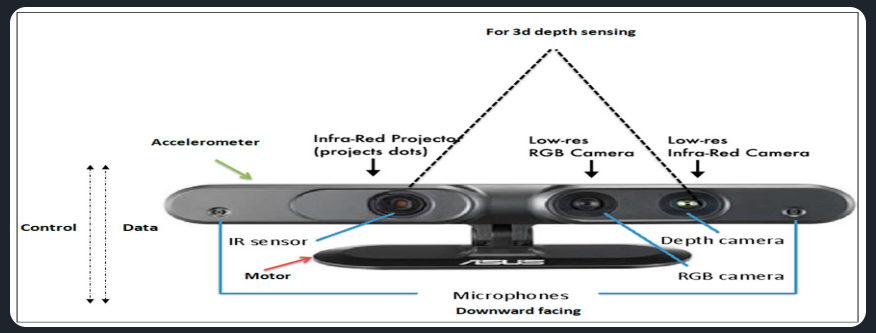
Interactive UI Elements

Dynamic buttons for each detected object + natural language command input interface

7	6+	3
Software Layers	ROS2 Topics	Unity Components

Hardware Setup

Component specifications and system configuration



Orbbec DaBai DCW

Type	RGB-D Depth Camera
Depth Technology	Structured Light
RGB Resolution	Up to 1920×1080
Depth Resolution	1280×800 @ 30fps
Interface	USB 3.0

FAIRINO Collaborative Arm

Payload Capacity **5-10 kg**	Reach **~1000 mm**
Degrees of Freedom **6 DOF**	Repeatability **±0.05 mm**

Safety Features: Force/torque sensing with collision detection, speed and separation monitoring, power and force limiting compliant with ISO/TS 15066 collaborative robot standards

Host PC

- Ubuntu 22.04 LTS
- WSL2 Compatible
- ROS2 Humble

Visualization

- Unity 3D Engine
- ROS-TCP-Connector
- Real-time Updates

Software Architecture

Layer 1-4: Perception and Calibration Pipeline

1 layer1_camera_check.py

Camera Stream Verification

Purpose: Initialize and verify RGB-D camera stream connectivity

Output: Confirmed /camera/color/image_raw and /camera/depth/points topics publishing at expected frame rates

2 layer2_detector_2d.py

YOLO Object Detection

Model: YOLOv8 (Ultralytics) with pretrained COCO weights

Filtered Classes: car, cell phone (confidence ≥ 0.3)

Output: Bounding boxes (x, y, w, h) + class labels + confidence scores published to /detections

3 layer3_position_3d.py

3D Position + Calibration Transform

Method: Sample depth values inside bounding box → IQR filter outliers → median calculation

Input: PointCloud2 from /camera/depth/points

Output: Camera-frame XYZ coordinates (meters) with timestamp synchronization

4 layer4_calibration.py

Hand-Eye Calibration (SVD)

Algorithm: Point correspondence SVD method for camera-to-robot transform

Process: Collect N point pairs → compute centroids → covariance matrix → SVD decomposition

Output: 4×4 homogeneous transformation matrix T (rotation + translation)

Layers 1-4 form the **perception and calibration pipeline**, transforming raw sensor data into robot-ready coordinates

Pipeline Latency: ~100-150ms

Software Architecture

Layer 5-7: Verification, Control, and Intelligence

5 layer5_calib_test.py

Calibration Verification

Purpose: Validate calibration accuracy by comparing predicted vs actual robot positions

Method: Move robot to test points, measure camera XYZ, apply transform, calculate error

Result: Mean reprojection error 16.16mm with 14 calibration points

6 layer6_commander.py

Robot Motion Execution

Motion Type: MoveCart with fallback from MoveL (singularity avoidance)

TCP Orientation: Fixed straight-down orientation (Rx=-177.293°, Ry=-2.013°, Rz=-121.047°)

Safety: ResetAllError() called before every move to clear alarm states

7 layer7_ai_agent.py

Natural Language AI Agent

Model: Gemini API with function calling capability

Tool Definition: move_to(object_name, speed) where speed ∈ [5, 50]%

Languages: Supports any natural language input (English, Korean, etc.)

Robot Motion Parameters

Fixed TCP Orientation

Rx=-177.293°, Ry=-2.013°, Rz=-121.047°

Straight-down approach for consistent picking

Hover Height Configuration

Table Z: -116.09mm | Hover: +150mm | Absolute: 33.91mm

Speed Range

5-50% of maximum velocity (user-controllable)

AI Agent Command Examples

English (Precise)

"go to phone with speed 20%"

Korean

"한드폰 있는데로 속도 48로 가라"

Vague (AI Decides)

"fly to red car, decide speed between 10 and 40"

Object Detection & 3D Positioning

YOLOv8 detection pipeline with depth-based 3D coordinate estimation

YOLOv8 Object Detection

Model Architecture

Ultralytics YOLOv8 with COCO pretrained weights

Input

RGB frame from /camera/color/image_raw

Output

Bounding boxes (x, y, w, h) + class labels + confidence scores

Filtered Classes

car, cell phone

Confidence Threshold

≥ 0.3

3D Position Estimation Algorithm

1. Depth Data Sampling

Sample depth values from PointCloud2 within bounding box region

2. Outlier Filtering (IQR)

Remove outliers using Interquartile Range method for robust depth estimation

3. Median Calculation

Compute median of filtered depth values for stable 3D position

4. Camera-Frame Output

Generate XYZ coordinates in camera reference frame (meters)

~100ms

Detection Latency

2

Object Classes

IQR + Median

Depth Filtering

Hand-Eye Calibration Algorithm

SVD-based point correspondence method for camera-to-robot transformation

SVD Calibration Process

1. **Collect Point Pairs**
   Move robot TCP to N known positions, simultaneously record camera XYZ coordinates at each position

2. **Compute Centroids**
   Calculate centroid_c for camera points and centroid_r for robot points

3. **Build Covariance Matrix**
   $H = \Sigma(\text{camera_i} - \text{centroid_c})(\text{robot_i} - \text{centroid_r})^T$

4. **SVD Decomposition**
   Decompose $H = U\Sigma V^T$ to obtain rotation and translation

5. **Extract Transform**
   $R = VU^T$, $t = \text{centroid_r} - R \cdot \text{centroid_c} \rightarrow 4\times4$ matrix T

Calibration Results

Points Used **14**

Calibration point pairs collected across workspace

Mean Reprojection Error **16.16 mm**

Average error between predicted and actual positions

Applied Offset Corrections

dX	dY
+56.85mm	+81.49mm

Mathematical Formulation

robot_xyz = T · camera_xyz

T = [R | t; 0 | 1] (4×4)

$R \in SO(3)$, $t \in \mathbb{R}^3$

AI Agent & Robot Motion Control

Gemini-powered natural language interface with precise motion execution

Gemini Function Calling

Tool Definition Schema

```
move_to(object_name, speed)
  object_name: "car" | "cell phone"
  speed: 5-50 (% velocity)
```

Command Processing Flow

Natural language → Gemini API → Function call extraction → Parameter validation → Robot execution

Response Generation

AI generates contextual replies published to /ai_reply topic for Unity display

Robot Motion Execution

MoveCart with Fallback

Primary motion command with automatic fallback from MoveL to avoid singularity errors (err=185)

Fixed TCP Orientation

Consistent end-effector approach angle:

Rx=-177.293°, Ry=-2.013°, Rz=-121.047°

Error Recovery

ResetAllError() called before every move to clear alarm state (err=14) and ensure reliable operation

Multilingual Command Examples

English (Precise)

"go to phone with speed 20%"

Direct command with explicit parameters

Korean

"한드폰 있는데로 속도 48로 가라"

Native language support via Gemini multilingual

Vague (AI Decides)

"fly to red car, decide speed between 10 and 40"

AI interprets intent and selects parameters

~1-2 sec

AI Command Processing

Any

Supported Languages

ROS2 Communication & Unity Visualization

Middleware backbone and real-time 3D interface

ROS2 Topic Map

/camera/color/image_raw
RGB frames from Orbbec camera
sensor_msgs/Image

/camera/depth/points
3D point cloud for depth estimation
sensor_msgs/PointCloud2

/detections
YOLO detection results with bounding boxes
std_msgs/String (JSON)

/robot_command
Motion commands to robot controller
std_msgs/String

/ai_command, /ai_reply
AI agent input/output messages
std_msgs/String

Unity 3D Components

DetectionSubscriber.cs
Subscribes to /detections topic, parses JSON bounding box data, and extracts object positions for visualization

DetectionUI.cs
Dynamic button generation for each detected object, 3D primitive spawning at calculated positions, real-time coordinate updates

AICommandUI.cs
Text input field for natural language commands, publishes to /ai_command, displays /ai_reply responses in UI

Coordinate System Transformation

Camera Frame

- Z: Forward (optical axis)
- X: Right
- Y: Down
- Units: meters

Robot Base Frame

- X: Forward
- Y: Left
- Z: Up
- Units: millimeters

$$robot_xyz = T \cdot camera_xyz$$

Launch Sequence

1. ros2 launch vision_robot_pick vision_robot.launch.py
2. Unity Editor → Press Play
3. System ready for detection & control

Results & Performance Metrics

Comprehensive system performance evaluation

16.16 mm

Calibration Mean Error

Reprojection error across 14 calibration points using SVD method

<1 mm

Robot XY Arrival Accuracy

Horizontal positioning vs predicted coordinates after calibration

5-10 mm

Physical Pick Accuracy

Real-world picking precision after offset correction (+56.85mm, +81.49mm)

~100 ms

Detection Latency

YOLOv8 inference time from image capture to bounding box output

1-2 sec

AI Command Processing

Gemini API response time including function calling and parameter extraction

Any Language

Multilingual Support

Gemini's multilingual capability enables commands in any natural language

Performance Summary

System Integration Success

All 7 software layers successfully integrated with ROS2 middleware. Camera, detection, calibration, and robot control operate as unified pipeline with ~250ms end-to-end latency.

AI Agent Effectiveness

Gemini function calling reliably extracts structured commands from natural language. Successfully tested with English, Korean, and vague commands requiring AI inference.

Known Issues & Solutions

Troubleshooting guide for common system challenges

MoveL Singularity Error

Error Code: 185

Symptom: Robot fails to execute linear motion command due to kinematic singularity (wrist joints aligned)

Solution: Use MoveCart instead of MoveL for Cartesian motion planning

MoveCart uses joint-space interpolation avoiding singular configurations

Robot Alarm State

Error Code: 14

Symptom: Robot enters alarm state after collision, E-stop, or previous error, blocking new commands

Solution: Call ResetAllError() before every motion command

Implemented in layer6_commander.py as automatic pre-move routine

Camera Depth All Zeros

Data Quality Issue

Symptom: Depth image contains all zero values, preventing 3D position estimation

Solution: Use /camera/depth/points instead of /camera/depth/image_raw

PointCloud2 format provides more reliable depth data than raw depth image

Unity Object Positioning

Coordinate System Mismatch

Symptom: Objects appear on wrong side in Unity visualization compared to real world

Solution: Negate Robot Y → Unity Z mapping

Different coordinate system conventions between ROS (right-handed) and Unity (left-handed)

Best Practice

All solutions have been integrated into the production codebase. Error handling is automatic in most cases, requiring no manual intervention during normal operation.

PROJECT CONCLUSION

Vision-Guided Robot Pick System

Successfully demonstrating the integration of **computer vision**, **robotic control**, and **natural language AI** for intuitive human-robot interaction in industrial automation

<1mm	7	Any	100ms
Positioning Accuracy	Software Layers	Language Support	Detection Latency

Source Code Available

github.com/prof-lijar/vision-guided-robot-pick-system